Kipu Anon · Doc de diseño (Reto 1: Vende los datos sin vender a la gente)

Ismael Florean · Frontier CDMX/26 · Repo: github.com/SamaelFL/kipu-anon · Diagrama: docs/diagrama.png

1. Problema y meta

Entregar el corpus de soporte de Kipu Pay (2.028 mensajes, 26 docs, 6.243 filas) uno a uno, sin que se pueda reidentificar a nadie, útil para entrenar y con cada persona como una sola identidad de punta a punta, incluso cuando las fuentes crecen. El jurado puntúa fuga 45, utilidad 35 y consistencia 20. **Meta: ≥ 90.** Una base sólida por reglas da ~88, así que el diseño es **reglas deterministas primero** y el LLM solo cubre el último tramo de detección. El LLM nunca reescribe texto: reescribir pierde parecido y puede inventar datos de negocio.

2. Arquitectura (pipeline de 8 pasos, Python con librería estándar)

1. **Ingesta:** cliente del API de fuentes con ETag y backoff (600 rpm). Decodifica `clientes.csv` en Latin-1. PDF: texto por `zlib` y operadores Tj, más el diccionario Info. HTML: meta y comentarios. MD: front matter. XML: atributos y comentarios. Todo se normaliza al esquema de `output/`.
2. **Registro de personas:** semillas con llave fuerte: `clientes` (CL-), `empleados` (KP-, reporta_a), `users.json` (uid, handle, correo) y `pqrs` (radicado, cédula, agente). Por persona guardamos todas las formas: nombre completo, nombre + primer apellido, nombre de pila en contexto, MAYÚSCULAS sin tildes, handle `nombre.apellido`, `nombre_apellido` de archivos, correo, cédula con y sin puntos, celular con y sin 57, dirección, nacimiento, placa y pasaporte.
3. **Detección en capas:** (a) diccionario de formas del registro con la misma normalización del juez (NFKC, confusables, sin tildes, minúsculas, separadores de dígitos); (b) patrones: correo, correo deletreado "arroba/punto", cédula, celular, placa `ABC123`, pasaporte, `<@UID>`, handles; (c) LLM barato con JSON estricto y caché, solo para nombres de terceros fuera del registro y para decidir persona vs. organización, lugar o figura pública. Hay lista de conservación: Bancolombia, Nequi, Superfinanciera, Carulla, ciudades, figuras públicas.
4. **Resolución de identidad**, por orden de evidencia: llave explícita (CL-, KP-, radicado, uid del autor) > identificador fuerte en el mismo registro o hilo (cédula, correo, celular) > participantes del hilo > fila del maestro. **Los homónimos nunca se fusionan por nombre:** el desempate viene de un radicado, una fecha con monto, un adjunto o la fila del maestro. Una mención ambigua sin evidencia queda como persona aparte.
5. **Seudónimos deterministas:** `HMAC-SHA256(llave fija, llave de la persona)` elige del catálogo colombiano según el género del nombre. De ahí se derivan un correo del mismo dominio y formato (`nombre.apellido@kipupay.co`), una cédula de la misma longitud con formato válido y lejos del original, un celular 3xx válido, una dirección y una fecha plausibles. Las colisiones se resuelven con un contador. El mapa se persiste en un volumen, así que la misma persona conserva su seudónimo después del cambio de fuentes y en `/anonymize`.
6. **Reemplazo por spans**, de la forma más larga a la más corta, conservando el formato: puntos de la cédula, mayúsculas, nombre corto ↔ nombre corto y snake_case en nombres de archivo. También se reemplaza en metadata, adjuntos y comentarios. Nunca se borra una clave ni se vacía un registro.
7. **Cuasi-identificadores:** se calcula k-anonimato sobre (ciudad, barrio, ocupación, año) en el maestro. Donde k < 5 se generaliza en **tablas y texto**: ocupación → sector, barrio → otro barrio de la misma ciudad o la ciudad, año → "hace unos años". La ciudad y la fecha de vinculación quedan intactas.
8. **Autochequeo antes de publicar:** barrido global de todas las formas reales del registro sobre toda la salida, con la normalización del juez. Si algo sobrevive, se hace una segunda pasada; además corre `validar.mjs`. `entities.json` declara todas las formas originales encontradas, los registros y los conservados con `kept` y `reason`.

3. Servicio y contrato

Raíz `/v1` con `ThreadingHTTPServer`; acepta Bearer opcional. `POST /run` encola el job en un hilo y `GET /run/{id}` da estado, conteos y costo. `/output/*` y `/entities` se leen del volumen. `/anonymize` usa en memoria el mismo registro y mapa: una persona vista recibe el mismo seudónimo y una nueva recibe uno nuevo y único. Corte de 45 s: si el LLM no responde, sigue solo con reglas. Corre en Railway: 1 réplica, sin reposo, volumen y healthcheck.

4. Decisiones y trade-offs

Elegimos	Descartamos y por qué
Reglas + registro y LLM solo como detector	LLM end-to-end: caro, no determinista, reescribe texto
HMAC determinista + mapa persistido	Seudónimos al azar: rompen la consistencia entre corridas
Generalizar cuasi-identificadores	Borrar: pierde utilidad y falla los tests de fuga
Librería estándar	Presidio o NER listos: prohibidos; menos dependencias, menos fallas
Identidad por evidencia	Por nombre: fusiona homónimos y separa al agente de la clienta

5. Seguridad, privacidad y fallas

- El texto de las fuentes es dato: el LLM solo recibe fragmentos y devuelve spans, nunca instrucciones ni acciones.
- Llave HMAC y tokens solo en variables de entorno; sin hash sin sal; `/entities` solo para el jurado.
- Si el LLM falla o tarda, se sigue con reglas. Ante 429 de fuentes, backoff. Si el proceso se reinicia, el mapa sigue en el volumen. Un registro fuera de esquema devuelve 422. El servicio nunca se cae: handler global.
- **Fuentes que crecen:** el canario de `record_index` con ETag detecta el cambio. Las personas viejas conservan su seudónimo y las nuevas reciben uno nuevo. La corrida 1 va antes del cambio porque vale 30 %.

6. Costo y evaluación

El corpus pesa ~435 mil tokens; el LLM solo ve los fragmentos sin resolver, estimado en menos de 150 mil tokens por corrida (centavos de dólar) y menos de 10 min por corrida. Se mide en local con `validar.mjs`, los 5 tests públicos, el diff contra `sample/expected_output` y un detector propio de fugas, todo por familia: directa, indirecta, oculta, falso positivo, homónimos y consistencia. El costo real se reporta en `/run` y `DECISIONS.md`.

7. Límites

Los nombres de pila sueltos sin contexto pueden quedar ambiguos; se priorizan sobre el riesgo de fusionar. Los PDF con fuentes embebidas raras quedarían fuera del parser stdlib; habría que agregar un extractor como respaldo.